1. Виконавець: Демчук Дмитро, група К-25. Лабораторна робота №4, варіант №13.

2. Умова (тільки те, що стосується варіанта № 13). Потрібна структура з двома цілими полями, якою одночасно користуються кілька потоків. З нею можна робити три речі: прочитати поле, записати в поле і отримати обидва поля одним рядком. Частоти операцій: для поля 0 читання 10% і запис 10%, для поля 1 читання 50% і запис 10%, запит рядка 20%.

3. Як захищені дані. У кожного поля є свій м'ютекс, тобто свій замок. Коли потік читає або змінює поле, він закриває тільки замок цього поля, тому дії над різними полями не чекають одна одну. Для рядка потрібні обидва числа одразу, тож береться обидва замки (через `std::scoped_lock`, який не дає потокам заблокувати один одного). Замки тримаються лише поки копіюються два числа, а сам рядок складається вже після цього, щоб решта потоків чекала менше. Кожне поле разом зі своїм замком лежить в окремому шматку пам'яті на 64 байти, щоб потоки не заважали одне одному через сусідні дані. Атомарні змінні та контейнери не використовувалися. Чому саме так: поля між собою не пов'язані, тому окремий замок на кожне поле дозволяє потокам одночасно працювати з різними полями. Один спільний замок змушував би чекати всіх, навіть коли потоки торкаються різних полів. Замки короткі за часом утримання і вирівняні по пам'яті, щоб потоки чекали якомога менше.

4. Результати. Кожен файл містить 3 000 000 операцій. Для двох потоків файл ділиться навпіл, для трьох на три частини, тому загальна робота завжди однакова. Кожен варіант запускався 10 разів, у таблиці середній час у мілісекундах. Файл спершу повністю читається в пам'ять, і лише потім вмикається таймер.

Файл	**1 потік**	**2 потоки**	**3 потоки**
За умовою (expected)	51,47	89,37	136,30
Рівні частоти (even)	54,45	97,71	140,42
96% рядків (mostly_string)	137,26	240,06	423,59

5. Висновки. Таблиця не збігається з очікуванням: я думав, що з більшою кількістю потоків буде швидше, а вийшло навпаки. На двох потоках робота триває в 1,7-1,8 раза довше, ніж на одному, на трьох у 2,6-3,1 раза. Ймовірна причина в тому, що кожна операція виконується майже миттєво, а 60% операцій звертаються до одного поля і до одного замка. Потокам доводиться чекати одне одного, а обмін даними між ядрами процесора коштує дорожче за саму операцію. Найдовше працює файл, де майже всі операції це запит рядка: треба створити рядок і взяти обидва замки. З одним потоком найшвидше, бо ніхто не заважає. Отже, для таких коротких операцій додаткові потоки не допомагають.

6. Що зроблено в роботі. Написано структуру `Fields` з двома замками і перетворенням у рядок. Написано програму, яка завантажує файли, ділить операції між потоками й рахує середній час. Додано перевірку файлів з повідомленнями про помилки та генератор трьох тестових файлів (за умовою, рівні частоти, 96% рядків). Перевірено код на гонки даних (ThreadSanitizer) і налаштовано збирання через CMake у режимі Release.